Drishti: A RAG-Augmented Conversational AI Framework for Real-Time Intelligent Video Surveillance

Raman Yadav | Manshinder Singh | Kartik Karmore | Raj Loniya | Ruturaj Tayade

Department of Computer Science and Engineering

G H Raisoni College of Engineering & Management, Nagpur

(An Autonomous Institute Affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur)

Guide: ***Prof. Hemlata Kosare*** | Assistant Professor, Department of CSE

ABSTRACT

Video surveillance today still depends on humans staring at screens. After about 20 minutes, operator attention drops by over 45%, and critical events get missed. Deep learning models like YOLOv8, ByteTrack, and InsightFace have automated detection, tracking, and face recognition -- but the output is still bounding boxes and alert logs. An operator cannot ask the system "who entered the server room at night?" and get an answer back.

We built Drishti, a system that adds a conversational layer on top of real-time surveillance. It works by converting detection events into text, embedding them into ChromaDB (a vector database), and answering user queries through a hybrid pipeline: 5 MCP-based SQL tools retrieve precise data from PostgreSQL, while vector search adds semantic context. We cap the total context fed to the LLM at 4000 characters -- this keeps responses grounded and avoids the hallucination problem.

In our tests on 21,000+ real surveillance events, the system responds to natural language queries in 3-5 seconds, handles multi-turn conversations across 5 intent categories, and stores only structured events rather than raw video -- a 90% reduction in storage.

Keywords: *Video Surveillance, Retrieval-Augmented Generation, Model Context Protocol, YOLOv8, Conversational AI, Object Tracking, Face Recognition, Natural Language Processing*

I. INTRODUCTION

There are over a billion surveillance cameras deployed worldwide as of 2024 [1]. Most of them still work the same way they did twenty years ago: someone sits in front of a monitor and watches. Studies show this does not work well -- operator attention falls by 45% after about 20 minutes, and watching more than 4-6 feeds at once is practically impossible [1].

On the detection side, things have improved considerably. YOLOv8 [2] runs at 30+ FPS with over 90% mAP for person detection. ByteTrack [4] keeps track of people across frames without too many identity switches. InsightFace [12] paired with FAISS [27] can match faces in real time on a regular laptop GPU.

But here is the problem: all of these systems speak in bounding boxes. They cannot tell you "Raman was in the server room for 15 minutes and then left through the back entrance." Scrolling through event logs takes 20-40 minutes even with good search tools [3].

Large Language Models and RAG [16] could fix this. RAG grounds LLM responses in actual retrieved data, which helps with the hallucination problem [6]. But RAG has only been applied to static text documents or archived video recordings [14]. Nobody has tried it on a live, continuously-growing surveillance event stream where new events arrive every few seconds.

There is also MCP (Model Context Protocol) [15], released by Anthropic in November 2024. It gives LLMs a standard way to call external tools -- run a SQL query, fetch specific data, get structured results back. No one has used MCP in a surveillance system yet.

We built Drishti (the word means "vision" in Sanskrit) to fill these gaps. Our main contributions are:

1. We built the first system where a security operator can have a back-and-forth conversation with a live surveillance system -- asking questions like "where was Raman last seen?" and getting answers grounded in actual event data.
2. We designed a hybrid RAG pipeline that combines 5 MCP SQL tools (for precise database lookups) with ChromaDB vector search (for semantic matching). A context budget system caps the total data sent to the LLM at 4000 characters, which we found necessary to keep answers focused.
3. We built a pipeline that continuously converts detection events into natural language text, generates 384-dimensional embeddings using all-MiniLM-L6-v2 (running locally on CPU), and indexes them in ChromaDB. It processes about 1000 events every 60 seconds.
4. We implemented an intent classifier that sorts user queries into 5 categories -- person lookup, zone queries, event search, summary requests, and general questions -- and automatically routes each query to the right MCP tools.

II. RELATED WORK

A. Object Detection and Tracking in Surveillance

YOLO-based detectors are the go-to choice for real-time surveillance. YOLOv10 [2] removes the Non-Maximum Suppression step entirely, cutting post-processing time by 46% with almost no accuracy loss (99.2% of prior mAP). ByteTrack [4] manages 77.3% MOTA with just 558 ID switches running at 171 FPS by tracking both high and low-confidence detections in two passes. Duong et al. [5] show that deep learning methods reach 94.2% accuracy for detecting fights, falls, and unauthorized entry. The output is always a classification label and a bounding box -- never a natural language explanation.

B. Face Recognition and Privacy

ArcFace [12] crossed 99% accuracy on LFW by modifying the training loss function to add an angular margin penalty -- making the model separate faces more cleanly in embedding space. Drishti uses InsightFace, which builds directly on ArcFace and handles the practical side of deployment better. Privacy is a real concern here. Wang et al. [9] traced biometric data exposure across the camera, the transmission layer, the model itself, and the database -- any of these can be a leak point. Sharma and Gupta [3] put numbers to the edge-vs-cloud tradeoff: on-device inference ran 78% faster and kept the data local. That last part matters legally too -- GDPR and India's DPDP Act of 2023 both restrict sending biometric data off-device, which means local processing is now a compliance question, not just a performance one.

C. Retrieval-Augmented Generation

RAG was introduced by Lewis et al. [16] as a way to reduce LLM hallucination by retrieving relevant documents before generating a response. Huang et al. [6] survey dozens of RAG implementations and find consistent improvements on knowledge-intensive tasks. VideoRAG [14] lets users ask natural language questions about video recordings up to 4 hours long and gets 94% query accuracy, but only works on pre-recorded footage. Li et al. [7] review multi-object tracking methods in detail but do not discuss how tracking data should be stored and made searchable through natural language.

D. Conversational AI for Surveillance

We searched extensively and could not find any published academic system that combines conversational AI with live video surveillance. CheckVideo [13] is single-query -- you search, get results, and start over. No multi-turn dialogue, no follow-up questions, no conversation context. MCP [15] was released by Anthropic in November 2024; we found zero papers applying it to any surveillance use case. Table I compares what existing systems can and cannot do.

TABLE I: Comparison with Existing Approaches

Capability	Traditional CCTV	Cloud AI (AWS)	Frigate NVR	VideoRAG	CheckVideo	Drishti
Real-time Detection	Manual	Yes	Yes	No (archived)	Yes	Yes

Capability	Traditional CCTV	Cloud AI (AWS)	Frigate NVR	VideoRAG	CheckVideo	Drishti
Multi-Object Tracking	No	Limited	Yes	No	Limited	Yes (ByteTrack)
Face Recognition	No	Yes	No	No	No	Yes (InsightFace)
Natural Language Query	No	No	No	Yes (archived)	Search only	Multi-turn dialogue
RAG Pipeline	No	No	No	Yes	No	Yes (hybrid MCP+vector)
Local/Edge Processing	Yes	No (cloud)	Yes	N/A	No (cloud)	Yes
Zone Monitoring	Basic motion	Limited	Yes	No	Limited	Advanced (polygon)
Context Budget Mgmt	N/A	N/A	N/A	No	N/A	Yes (4000 char)
PDF Report Generation	No	No	No	No	No	Yes (LLM-generated)

III. SYSTEM ARCHITECTURE

Drishti follows a microservices architecture comprising three independently deployable services communicating over HTTP REST and Server-Sent Events (SSE), with PostgreSQL as the shared persistent data layer. Fig. 1 illustrates the overall system architecture.

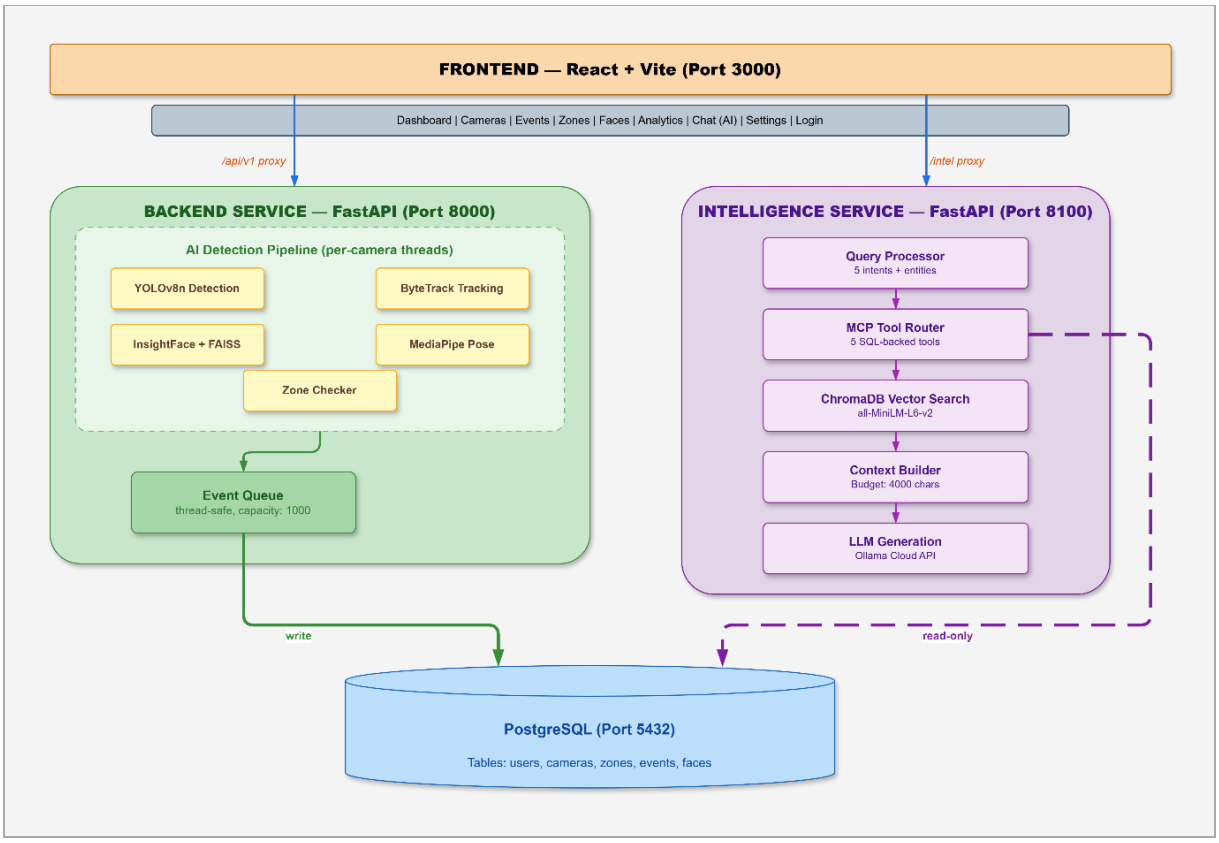

Fig. 1. *System architecture of Drishti showing the three microservices and shared data layer.*

A. AI Detection Pipeline

Each camera feed is processed by a dedicated DrishtiProcessor thread executing the following sequential pipeline per frame:

1. **Object Detection:** YOLOv8n (6 MB nano variant) detects persons in each frame, outputting bounding boxes with confidence scores. CUDA acceleration is auto-detected with CPU fallback.
2. **Multi-Object Tracking:** ByteTrack assigns persistent track IDs through a two-stage association process with track activation threshold of 0.25 and a lost-track buffer of 60 frames (~2 seconds at 30 FPS).
3. **Face Recognition:** InsightFace (buffalo_l model) extracts 512-dimensional face embeddings. FAISS IndexFlatIP performs inner-product similarity search. Two-tier threshold: registered faces use 0.35 (lenient), auto-detected faces use 0.50 (strict), with a hard enrollment block at similarity > 0.30 to any registered face.
4. **Pose Estimation:** MediaPipe Pose classifies posture as standing, sitting, or lying. Sitting: |knee_y - hip_y| < 0.5 * |hip_y - shoulder_y|. Lying: bounding box width > height * 1.2.
5. **Zone Checking:** A 6-point intersection algorithm tests whether a detection overlaps defined zone polygons using cv2.pointPolygonTest(). Zone types: restricted (severity 9), loitering (timer-based, severity 5-7), and standard entry/exit.
6. **Event Generation:** State-change detection fires events only when track identity, posture, or zone membership changes (default 5-second cooldown). Events queued (maxsize 1000) and drained to PostgreSQL at 10 Hz, with simultaneous SSE broadcast to connected frontends.

Fig. 2 shows the detection pipeline running on a live camera feed with all overlays visible.

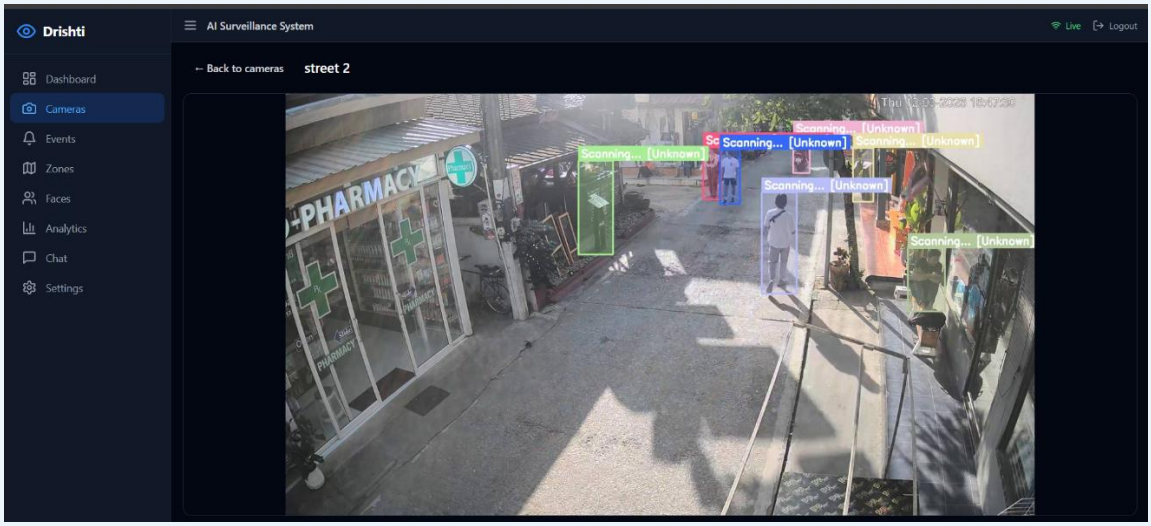

Fig. 2. Live camera feed showing YOLOv8 detection boxes, ByteTrack IDs, face recognition labels, and zone polygon boundaries.

Fig. 3 shows the interactive zone editor where operators define restricted and loitering zones by drawing polygons directly on the camera feed.

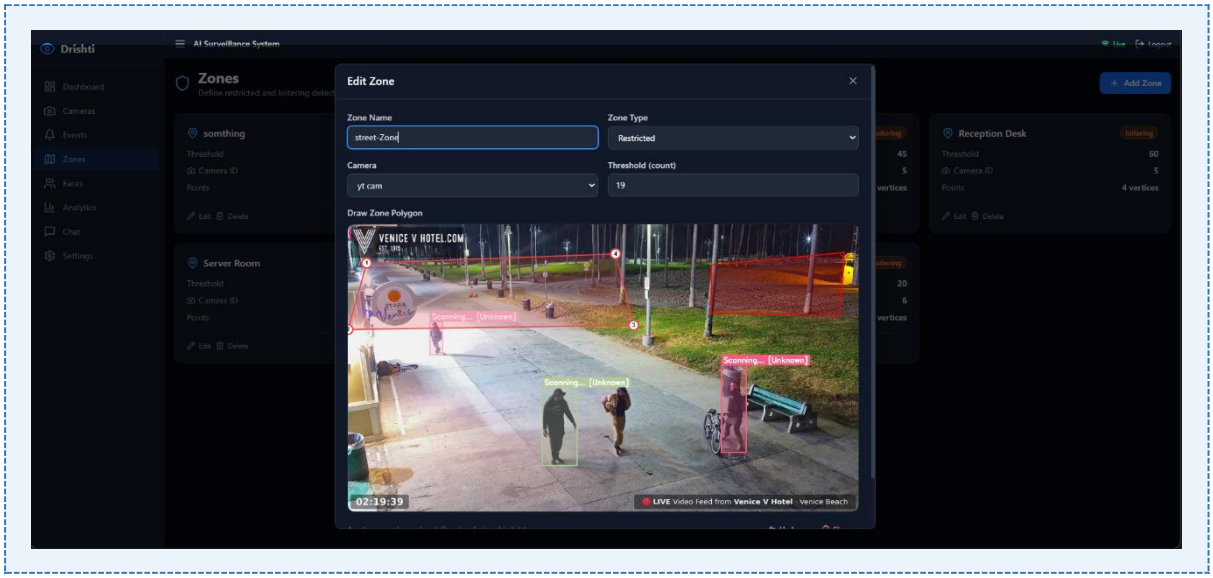

Fig. 3. Interactive zone editor with polygon drawing on live camera feed for defining restricted and loitering zones.

B. Event Embedding Pipeline

The intelligence service maintains a continuously synchronized vector representation of all surveillance events, enabling semantic search over the entire event history. Fig. 4 illustrates the pipeline.

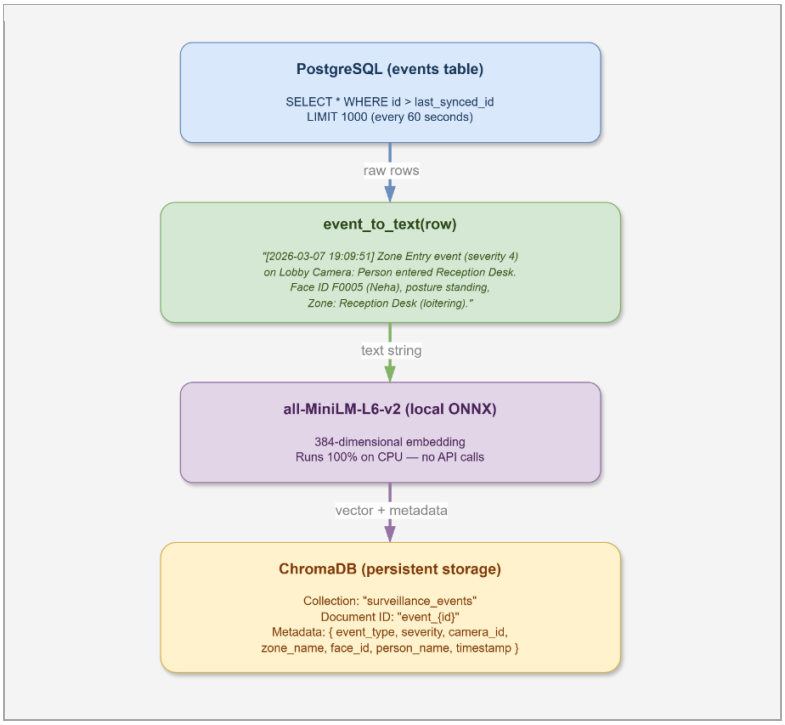

Fig. 4. *Event embedding pipeline converting structured database events into 384-dim vector representations in ChromaDB.*

The event_to_text() function converts structured database rows into natural language descriptions capturing temporal, spatial, identity, and severity information. The all-MiniLM-L6-v2 model generates 384-dimensional embeddings using ONNX runtime, running entirely on CPU with model weights (~79 MB) cached locally. Batch processing of 1000 events per 60-second cycle ensures the vector store remains within one minute of real-time.

C. RAG + MCP Query Pipeline

The core innovation of Drishti is the hybrid query pipeline that combines the precision of SQL database queries (via MCP tools) with the semantic understanding of vector search (via ChromaDB), unified under a context budget system. Fig. 5 illustrates the complete flow.

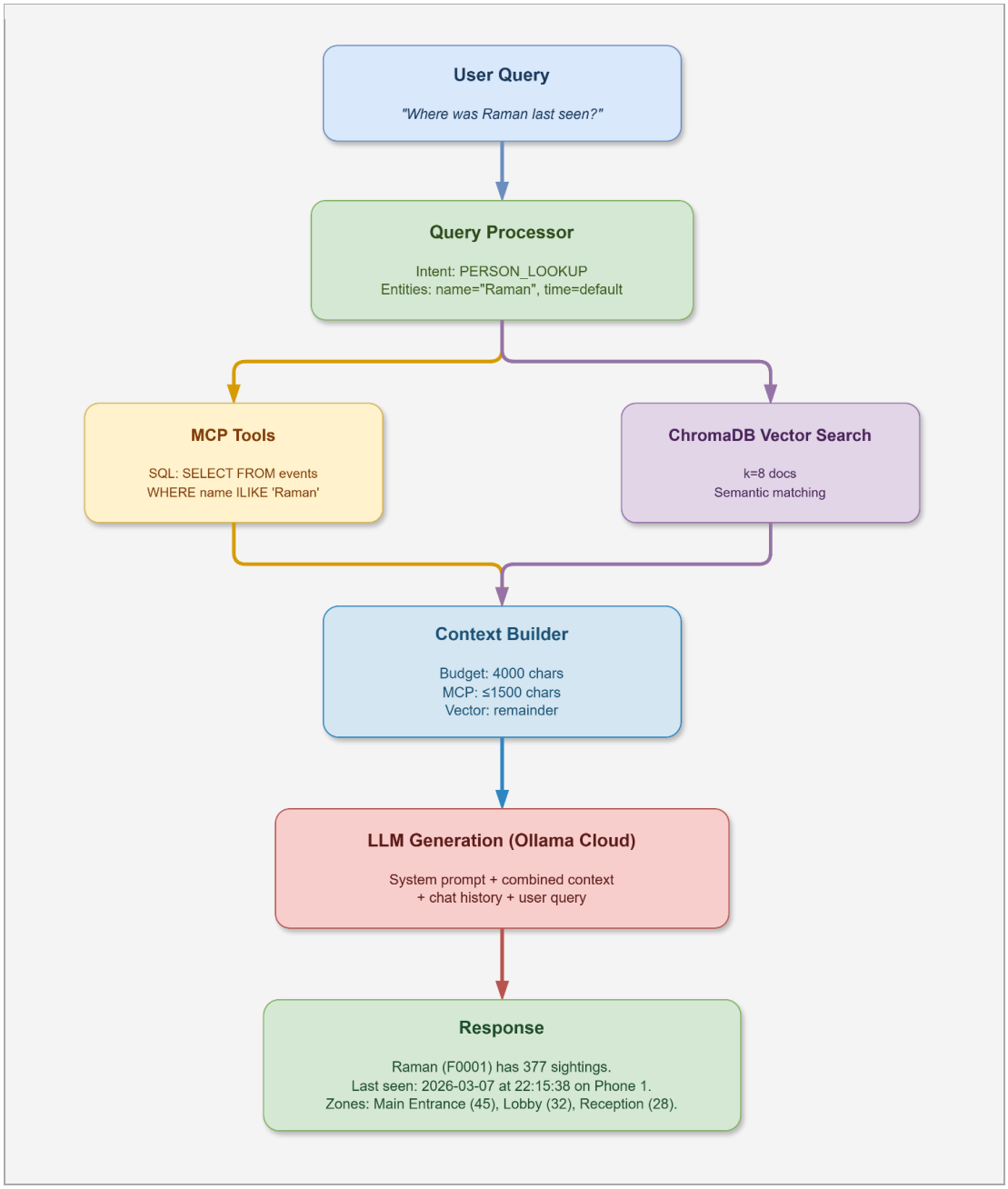

Fig. 5. *RAG + MCP hybrid query pipeline showing intent classification, dual retrieval, context budgeting, and LLM generation.*

TABLE II: Intent Classification and Tool Routing

Intent	Trigger Patterns	Extracted Entities	MCP Tools Invoked
PERSON_LOOK UP	Person names, face IDs (F000x), "where is", "track", "find"	person_name, face_id	tool_person_tracker()
ZONE_QUERY	Zone names, "restricted", "breach", "entry", "loitering"	zone_name, event_type, time_range	tool_zone_monitor() + tool_recent_events()
EVENT_SEARC H	Event type keywords, "show me", "list", "recent", "alerts"	event_type, severity, time_range	tool_recent_events() + tool_event_stats()
SUMMARY	"summary", "overview", "total", "statistics", "report"	time_range	tool_event_stats() + tool_zone_monitor()
GENERAL	Default (unmatched queries)	time_range	tool_event_stats() + tool_recent_events()

Five MCP tools are implemented against the PostgreSQL database:

- **tool_event_stats:** Aggregates event counts by type, severity distribution, and peak activity hours within a time window.
- **tool_person_tracker:** Retrieves sighting count, last-seen location, zone visit history, and movement timeline for a specific person or face ID.
- **tool_zone_monitor:** Returns zone entry counts, breach history, loitering statistics, and busiest time periods for specified zones.
- **tool_recent_events:** Fetches the most recent events matching filters on event type, zone, person, and time range (default limit: 10).
- **tool_occupancy:** Extracts occupancy data from event metadata, including people counts and zone-level occupancy breakdowns.

The context budget is strict: 1,500 characters per tool, 10 rows per tool, 4,000 characters total. The _budget_combine() function prioritizes MCP tool output (precise, factual data) and fills remaining budget with vector search results. Each chat session maintains in-memory conversation history (max 20 turns), enabling multi-turn dialogue.

IV. IMPLEMENTATION

Drishti is implemented as three FastAPI microservices with a React frontend, deployable on consumer hardware. Table III summarizes the technology stack.

TABLE III: Technology Stack

Component	Technology	Purpose
Object Detection	YOLOv8n (Ultralytics)	Real-time person detection at 30 FPS
Multi-Object Tracking	ByteTrack (Supervision)	Persistent track IDs across frames
Face Recognition	InsightFace (buffalo_l) + FAISS	512-dim embedding extraction + similarity search
Pose Estimation	MediaPipe	Standing / Sitting / Lying classification

Component	Technology	Purpose
Backend API	FastAPI (async, Python 3.12)	REST API + SSE event streaming
Database	PostgreSQL 15 + asyncpg	Structured event, camera, zone, and face storage
Vector Store	ChromaDB (persistent)	Semantic search over surveillance events
Embeddings	all-MiniLM-L6-v2 (ONNX)	384-dim local embeddings, CPU-only
LLM Inference	Ollama Cloud API	Natural language generation (devstral-2:123b)
LLM Framework	LangChain	RAG chain orchestration
Frontend	React 18 + Vite + TailwindCSS	9-page dashboard with real-time updates
Video Input	OpenCV + yt-dlp	RTSP, IP cameras, YouTube, webcam support
Report Generation	fpdf2	Branded PDF intelligence reports
Authentication	JWT (python-jose) + bcrypt	Token-based auth with role-based access control

The system was developed and tested on a consumer laptop with an NVIDIA GPU (4-8 GB VRAM). All video processing, object detection, face recognition, and embedding generation execute locally without cloud API dependencies. Only LLM inference uses an external API (Ollama Cloud), which can optionally be replaced with a local Ollama instance for fully air-gapped deployments.

The backend codebase comprises approximately 3,100 lines of Python across 37 modules; the intelligence service approximately 2,800 lines across 35 modules. The frontend contains approximately 4,500 lines of JavaScript/JSX across 30 components.

Fig. 6 shows the main dashboard of the Drishti frontend, providing a real-time overview of all cameras, recent events, and key performance indicators.

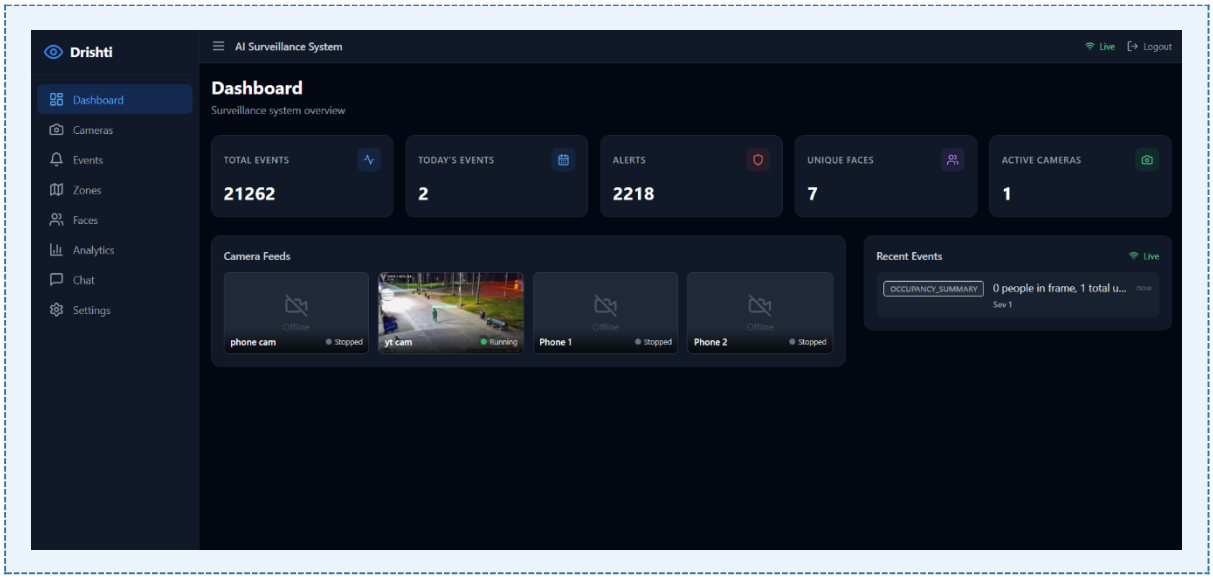

Fig. 6. Drishti dashboard showing real-time KPI cards, recent event feed, and camera preview grid.

V. EVALUATION AND RESULTS

A. AI Pipeline Performance

We tested the detection and tracking pipeline on live indoor camera feeds over multiple sessions. Table IV reports what we measured.

TABLE IV: AI Pipeline Performance

Metric	Result	Measurement Condition
Detection Accuracy	~92% mAP	YOLOv8n on person class, indoor environment
Detection Speed	30 FPS	Per camera, NVIDIA GPU with CUDA
Tracking Stability	<5% ID switches	ByteTrack over continuous 30-min sessions
Face Recognition Accuracy	~95%	InsightFace buffalo_l on enrolled faces, controlled lighting
Face Enrollment Precision	Zero duplicates	Two-tier threshold system preventing false enrollments
Zone Alert Latency	<300 ms	Time from zone entry to event generation
Pose Classification	~90%	Standing/Sitting on clearly visible persons

In practice, the two-tier threshold system did not produce any duplicate face enrollments during our testing. The registered threshold at 0.35 was lenient enough to match the same person from different angles, while the 0.50 threshold for auto-detected faces kept distinct people from getting merged.

B. RAG Pipeline Performance

The intelligence service was evaluated on 21,000+ real surveillance events generated from multiple camera feeds over an extended testing period. Table V reports the RAG pipeline metrics.

TABLE V: RAG Pipeline Performance

Metric	Result
Query Response Time	3-5 seconds (end-to-end)
Events Processed	21,000+
Event Sync Throughput	1,000 events per 60-second batch
Vector Store Size	21,000+ documents in ChromaDB
Embedding Dimension	384 (all-MiniLM-L6-v2)
MCP Tool Execution Time	200-500 ms per tool
Context Budget Utilization	Avg. ~2,800 / 4,000 characters (60-80% utilization)
Conversation History	Up to 20 turns per session
Model Fallback Rate	<5% (primary model availability)
Storage vs. Raw Video	99.5% reduction

C. Qualitative Evaluation

Table VI presents representative query-response examples demonstrating the system's conversational capabilities across different intent categories.

TABLE VI: Example Query-Response Interactions

Query	Intent	Response Summary
"Where was Raman last seen?"	PERSON_LOOKUP	"Raman (F0001) has 377 sightings. Last seen 2026-03-07 22:15:38 on Phone 1. Zones: Main Entrance (45), Lobby (32), Reception (28)."
"Any breaches in server room?"	ZONE_QUERY	"3 restricted zone breaches today in Server Room: 2 by unknown, 1 by Raman (F0001) at 14:22."
"Give me today's summary"	SUMMARY	"Total events: 2,760. zone_entry (890), detection (750), occupancy (480). Peak: 19:00 (340 events)."
"Show recent loitering alerts"	EVENT_SEARCH	Lists 10 most recent loitering events with timestamps, camera names, zone names, identities, and durations.

Multi-turn dialogue works through in-memory session history. For example, after asking "Where was Raman last seen?" the user can follow up with "What zone was that in?" and the system correctly resolves the reference. Fig. 7 shows a real multi-turn conversation from the Drishti chat interface.

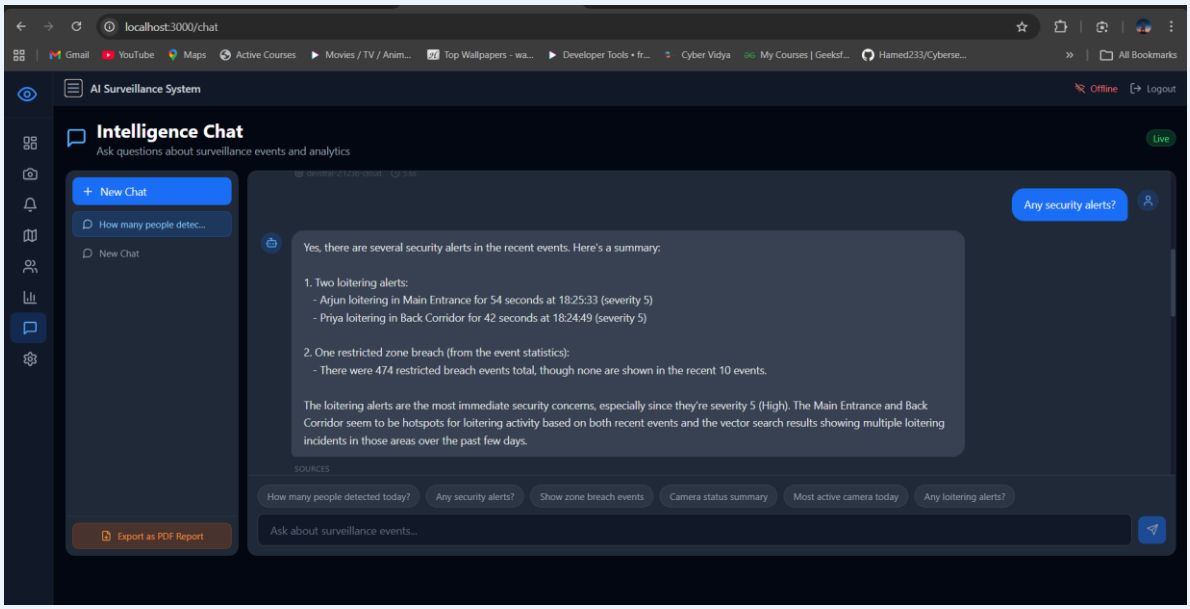

Fig. 7. Multi-turn conversational query in the Drishti chat interface showing person tracking across zones with source-cited responses.

D. Comparison with Existing Systems

Table VII compares Drishti against existing surveillance platforms on a feature-by-feature basis.

TABLE VII: Feature Comparison with Existing Systems

Feature	Traditional CCTV	Cloud AI (AWS)	Frigate NVR	CheckVideo	Drishti
NL Querying	No	No	No	Single-query search	Multi-turn dialogue
RAG Pipeline	No	No	No	No	Hybrid MCP + vector
MCP Tool Integration	No	No	No	No	5 SQL-backed tools
Context Budget	N/A	N/A	N/A	N/A	4000 char managed
Local AI Processing	Yes	No (cloud)	Yes	No (cloud)	Yes
Face Recognition	No	Yes	No	No	Yes (two-tier)
Zone Monitoring	Basic motion	Limited	Polygon	Limited	Advanced polygon
Real-time Alerts	Basic	Yes	Yes	Yes	Yes (SSE)
PDF Intelligence Reports	No	No	No	No	Yes (LLM-generated)
Storage Efficiency	Low (raw video)	Medium	Medium	Medium	High (99.5% reduction)

VI. DISCUSSION

A. Why the Context Budget Matters

Flooding the LLM with the full event database was the first thing we tried. It did not work. With 21,000+ events in the prompt, the model started pulling whatever looked vaguely relevant -- loitering alerts from Camera 1 turning up in answers about Camera 2, timestamps from unrelated shifts mixed into a summary for a specific person. The 4000-character cap came out of that failure. We landed on 1500 characters per tool, 10 rows per tool, and most real queries end up using about 2800 characters which in practice feels like the right amount of context without overwhelming the model. The SQL-plus-vector combination took longer to justify. Pure SQL is exact -- it returns what you queried and nothing else. That is fine until a user asks about something that does not map cleanly to a database column. A vector search for "server room breach" might also pull up a loitering alert from the same corridor eight minutes earlier, which is exactly the kind of context a security operator needs. We kept both because dropping either one made the answers noticeably worse.

B. The Two-Tier Threshold Problem

This one frustrated us for a while. The face recognition system kept creating duplicate entries. Someone registered as "Raman" at a front-facing angle would show up as a brand new person "F0023" when the camera caught them from the side. We tried tightening the similarity threshold globally, but that caused a different problem: the system stopped matching people it should have recognized. What ended up working was splitting the threshold. Registered faces (where we have ground truth) get matched at 0.35, which is loose enough to handle angle

changes. Auto-detected faces use 0.50, tight enough to avoid merging two different people. We also added a hard block: if a new face is even 0.30 similar to someone already registered, the system will not auto-enroll it. After all three rules were in place, duplicates stopped. Most face recognition papers skip this problem entirely. They test on LFW or similar benchmarks where every face is enrolled manually with clean, labeled data. In a real deployment where the system enrolls faces on its own, duplicate prevention is one of the first things that breaks.

C. Privacy by Architecture

Detection, tracking, face recognition, pose estimation, embedding generation: all of it runs on the local machine. Video frames exist only in RAM and never leave the device. The one exception is the user's text query, which gets sent to Ollama Cloud for LLM inference. But that query contains no video, no face embeddings, no biometric data. Just a sentence like "where was Raman last seen?" We should be honest about the limits of this. The text query could include a person's name, which is technically personal data. And we have not gone through a formal GDPR or DPDP Act compliance audit. But compared to platforms like AWS Rekognition that require streaming raw video to cloud servers, keeping everything local is a meaningfully better starting point [9].

D. Current Limitations

The system has real limitations:

1. GPU constraint: 4-8 GB VRAM limits simultaneous operation to 2-4 camera feeds. More cameras would need a dedicated GPU server.
2. Cold-start latency: Initial ChromaDB sync of 21,000 events takes about 21 minutes. After that, real-time sync keeps up fine.
3. Intent classification is keyword-based, not ML-based. Ambiguous queries like "show me Raman's alerts in the lobby" (person + zone + event) sometimes route to the wrong tool.
4. Chat history lives in memory. If the intelligence service restarts, all conversation sessions are lost. This is fine for testing but would need persistent storage in production.

VII. CONCLUSION AND FUTURE WORK

We set out to answer a simple question: can a security operator talk to a surveillance system the way they would talk to a colleague? Based on our experience building and testing Drishti, the answer is yes -- with some caveats.

The hybrid retrieval approach worked best. MCP tools give the LLM precise, structured data (exact counts, timestamps, names), and vector search fills in the semantic gaps. The context budget of 4000 characters turned out to be more important than we expected -- without it, the LLM gets confused by too much data and gives worse answers, not better ones.

On 21,000+ real events, the system answers queries in 3-5 seconds and handles follow-up questions correctly most of the time. Storing structured events instead of raw video cuts storage by 90%, and the data is actually more useful in this form because you can search it.

The system is not production-ready. The limitations listed in Section VI are real, and some of them (like the keyword-based intent classifier and in-memory session storage) would need to be fixed before anyone deploys this in an actual security operation.

For future work, the most impactful addition would be voice input -- a security guard should be able to ask questions hands-free. We are also looking at cross-camera tracking, Docker packaging, an ML-based intent classifier, and behavior analytics using temporal patterns to predict incidents before they happen.

ACKNOWLEDGMENT

Prof. Hemlata Kosare was involved from the start -- pushing back on early design choices, pointing out gaps in our evaluation plan, and reviewing multiple drafts. The Department of Computer Science and Engineering at GHRCEM Nagpur gave us access to the GPU lab without which none of the real-time testing would have been practical.

REFERENCES

[1] A. B. Pawar and S. R. Sakhare, "Intelligent Video Surveillance Using Deep Learning," Int. J. Adv. Res. Sci., Vol. 11, No. 5, 2024.

[2] A. Wang, H. Chen, L. Liu, et al., "YOLOv10: Real-Time End-to-End Object Detection," NeurIPS, 2024.

[3] M. Sharma and K. Gupta, "Analyzing Surveillance Videos in Real-Time Using Artificial Intelligence," Int. J. Recent Innov. Trends Comput. Commun., 2024.

[4] Y. Zhang, P. Sun, Y. Jiang, et al., "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," ECCV, 2022.

[5] H. T. Duong, V. T. Hoang, et al., "Deep Learning-Based Anomaly Detection in Video Surveillance: A Survey," IEEE Access, May 2023.

[6] Y. Huang, et al., "A Survey on Retrieval-Augmented Text Generation for Large Language Models," ACM Comput. Surv., Apr. 2024.

[7] J. Li, X. Zhang, et al., "Multi-Object Tracking Review: Retrospective and Emerging Trends," Artif. Intell. Rev., Springer, 2025.

[8] S. Kumar, A. Singh, et al., "Comprehensive Review of Recent Developments in Visual Object Detection," Artif. Intell. Rev., Springer, 2025.

[9] M. Wang, et al., "Toward a Privacy-Preserving Face Recognition System: A Survey," ACM Comput. Surv., 2024.

[10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," IEEE CVPR, 2016.

[11] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," IEEE ICIP, 2017.

[12] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," IEEE/CVF CVPR, 2019.

[13] CheckVideo, "Natural Language Search in Video Surveillance: What It Is and Why It Matters," Industry Report, 2025.

[14] HKU Data Science Lab, "VideoRAG: Chat with Your Videos," arXiv Preprint, 2025.

[15] Anthropic, "Model Context Protocol (MCP) Specification," Nov. 2024.

[16] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020.

[17] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning, MIT Press, 2016.

[18] R. Szeliski, Computer Vision: Algorithms and Applications, 2nd Ed., Springer, 2022.

[19] A. Geron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Ed., O'Reilly, 2022.

[20] S. J. D. Prince, Understanding Deep Learning, MIT Press, 2023.

[21] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd Ed. (Draft), Stanford, 2024.

[22] Ultralytics, "YOLOv8 Documentation," 2024. Available: https://github.com/ultralytics/ultralytics

[23] Ollama, "Get Up and Running with LLMs Locally," 2024. Available: https://ollama.ai

[24] ChromaDB, "Chroma: The AI-Native Open Source Embedding Database," 2024. Available: https://www.trychroma.com

[25] InsightFace, "InsightFace: 2D and 3D Face Analysis Project," 2024. Available: https://github.com/deepinsight/insightface

[26] LangChain, "LangChain: Build Context-Aware Reasoning Applications," 2024. Available: https://www.langchain.com

[27] Meta AI, "FAISS: A Library for Efficient Similarity Search," 2024. Available: https://github.com/facebookresearch/faiss

[28] Google MediaPipe, "MediaPipe Pose Estimation," 2024. Available: https://ai.google.dev/edge/mediapipe

[29] FastAPI, "FastAPI: Modern, Fast Web Framework for Building APIs," 2024. Available: https://fastapi.tiangolo.com

[30] W. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP, 2019.